to managing an ongoing desired application state.

Service: This exposes an application running on a set of pods as a network service.

ClusterIP: This component exposes the service on a cluster-internal IP. Choosing this value makes the service reachable only from within the cluster (default).

NodePort: This exposes the service on each node's IP at a static port (the NodePort). A ClusterIP service, to which the NodePort service will route, is automatically created.

Ingress: This is the primary method of exposing a cluster service to the outside world. These are load balancers or routers that usually offer secure sockets layer (SSL) termination, name-based virtual hosting, etc.

Volume: This is storage that is tied to the pod life cycle, consumable by one or more containers within the pod.

Persistent volume (PV): This represents storage resources in the cluster. It is the 'physical' volume on the host machine that stores your persistent data.

PersistentVolumeClaim (PVC): This fulfills the request for storage by the user, where it claims the resources from the PersistentVolume.

Figure 2 depicts the flow of the request to the deployed application over Kubernetes. The various components of Kubernetes come together to facilitate an environment for the services. The NodePort exposes the static port at which the traffic is intercepted, and based on the request the Ingress segregates it. Ingress filters the request and forwards it to the appropriate service, which is sitting on the top of deployment in the Kubernetes cluster. A deployment can have multiple replicas of a pod, which is responsible for the business logic and data retrieval/modification. Data-based services are handled by the volume associated with the pod. The cluster administrator provisions the infrastructure storage.

Kubernetes container runtime

Kubernetes is attached to a container runtime, which is responsible for the life cycle of the containers on Kubernetes nodes while also managing container images and their pods. Container runtime is also responsible for container interactions such as attach, exec, ports, logs, etc. With the release of Kubernetes version 1.5, the Kubernetes community introduced the Container Runtime Interface (CRI). This is a new plugin API for container runtime in Kubernetes which connects container runtime with kubelet.

With its implementation as a separate entity, it allows users to switch out container runtime implementations instead of having them built into the kubelet. As shown in Figure 3, kubelet communicates with the container runtime; it bridges that communication via the use of gRPC (Google remote procedure call), where kubelet acts as a client and the CRI shim acts as the server. A shim is a library that intercepts various API calls and then either redirects the operations elsewhere or handles them itself. In this routine, the shim can change the passed arguments and facilitates communication. Some of the common container runtimes with Kubernetes are containerd, CRI-O, and Docker.

Setting up a Kubernetes cluster using minikube

minikube is a locally deployed single-node Kubernetes cluster environment that can be used as a learning or development environment. It can be

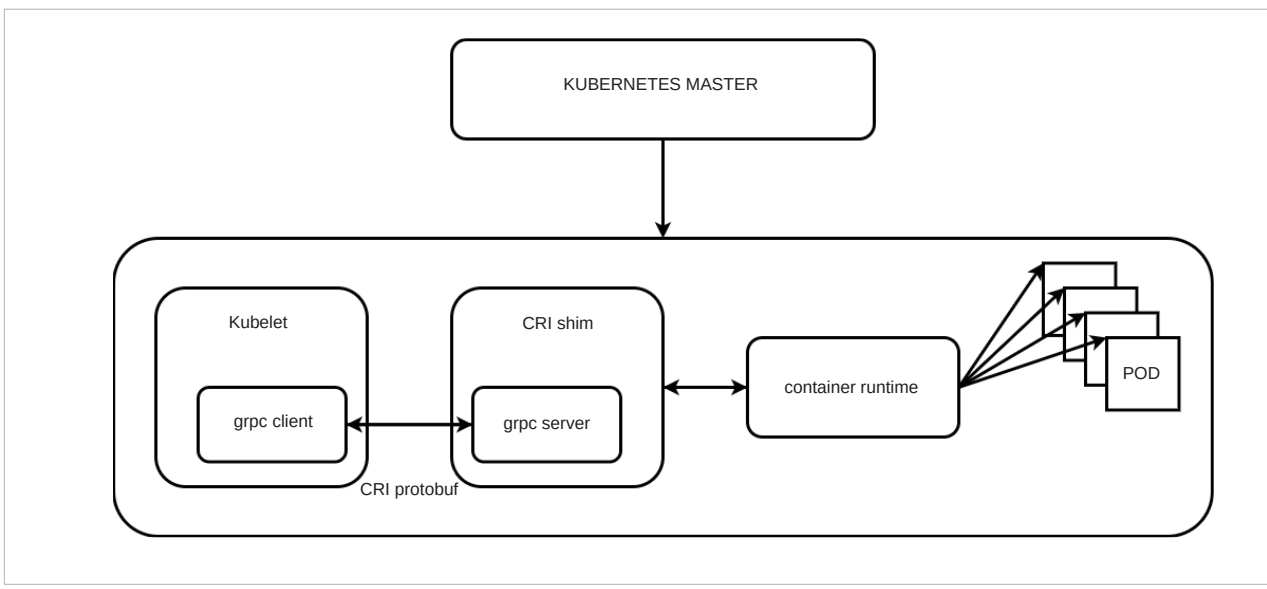

Figure 3: Various components of container runtime in Kubernetes worker node